



Universidad Nacional Experimental de Guayana

Vicerrectorado Académico

Coordinación General de Pregrado

Proyecto de carrera: Ingeniería en Informática

Unidad Curricular: Lenguajes y compiladores

INTRODUCCION A LOS LENGUAJES Y COMPILADORES.

Profesor:

Marquez, Félix

Autores:

Longart, Daniela V-31445710.

Martínez, Fabiana V-30498516.

Ortiz, Sebastián V- 30576350.

Valencia, Haddan V-31818222.

Ciudad Guayana, 14 de mayo del 2026.

Introducción

En el panorama actual de la computación, la brecha entre el pensamiento lógico humano y la ejecución binaria en el silicio se ha vuelto cada vez más compleja y sofisticada. Lo que alguna vez fue un proceso lineal de traducción de código, se ha transformado en un ecosistema de herramientas críticas que permiten no solo ejecutar software, sino garantizar su seguridad, optimizar su rendimiento y recuperar lógica perdida en sistemas heredados.

El presente informe, desarrollado por el equipo de T-Code Solutions, se propone desglosar y analizar las tres piezas fundamentales de esta arquitectura de traducción: los Metacompiladores, los Intérpretes (con sus modernas arquitecturas híbridas) y los Decompiladores.

A lo largo del documento presente, se va a explorar cómo estas herramientas, aunque operan en direcciones o tiempos distintos, comparten una estructura común como, el manejo de gramáticas formales y estructuras de datos abstractas. El informe no se limita a la teoría académica; se vincula cada concepto con la realidad tecnológica del 2026.

El objetivo es demostrar que comprender estos mecanismos no es solo un ejercicio de fundamentos de programación, sino una necesidad estratégica para cualquier ingeniero que busque construir el "puente" hacia soluciones tecnológicas soberanas, seguras y eficientes.

Desarrollo

Metacompiladores

Metacompiladores (El Generador): es una herramienta o software que genera compiladores. Es en esencia, un compilador para compiladores. En lugar de traducir un lenguaje de programación como código ejecutable, un metacompilador permite especificar y generar compiladores para nuevos lenguajes o versiones personalizadas de lenguajes ya existentes.

Ahora bien, la diferencia clave entre un compilador convencional y un metacompilador es que este último fue creado con la finalidad de traducir directamente los programas de un lenguaje de alto nivel a código máquina o intermedio. En su lugar, proporciona un marco para definir un nuevo lenguaje y genera el compilador correspondiente a partir de esa especificación.

Un metacompilador tiene tres partes principales:

- Especificación del lenguaje fuente: define la gramática (usando CFG o BNF).
- Definición semántica: describe el comportamiento de las construcciones del lenguaje.
- Generador de código: traduce el código fuente a código ejecutable o intermedio (ensamblador, bytecode, etc.).

Tipos de Meta compiladores:

- Metacompiladores basados en gramáticas: Se centran fundamentalmente en la definición formal de la gramática del lenguaje fuente, generalmente usando notaciones como BNF. A partir de esa especificación, generan automáticamente un analizador sintáctico (parser). Un ejemplo clásico es Yacc, que toma una gramática y produce un compilador parcial.

- Metacompiladores de propósito general: Son más flexibles y potentes, ya que no solo permiten definir gramáticas, sino también especificar la semántica completa del lenguaje, reglas de optimización y estrategias de generación de código. Ofrecen lenguajes de definición más ricos. Un representante destacado es ANTLR, muy usado para construir lenguajes complejos y analizadores.
- Metacompiladores orientados a la sintaxis concreta: En lugar de trabajar únicamente con gramáticas abstractas, estos se enfocan en el procesamiento de cadenas y autómatas de estados. Son ideales para tareas de reconocimiento léxico, validación de patrones o procesamiento de protocolos. Un ejemplo es Ragel, que permite incrustar máquinas de estados en el código generado.

¿Cómo comparten con los compiladores el uso de herramientas como ANTLR o

Lex/Yacc para automatizar el análisis léxico y sintáctico?

1. Base común: autómatas y gramáticas

- Análisis léxico: se apoya en autómatas finitos (generados a partir de expresiones regulares).
- Análisis sintáctico: se apoya en gramáticas libres de contexto y algoritmos de parsing (LL, LR, LALR, etc.).

Tanto un compilador construido a mano como uno generado por un metacompilador necesitan estas dos fases. Para no implementarlas desde cero, se emplean herramientas como Lex/Flex, Yacc/Bison o ANTLR.

2. Uso directo en compiladores manuales

Un desarrollador que escribe un compilador de forma tradicional puede usar Lex para generar el analizador léxico y Yacc para generar el analizador sintáctico. Luego él mismo

escribe el resto del compilador (análisis semántico, generación de código). Aquí las herramientas actúan como asistentes de automatización para las fases más mecánicas.

3. Uso interno o como salida en metacompiladores

Un metacompilador (como Yacc, ANTLR o Coco/R) es una herramienta que, a partir de una especificación del lenguaje (gramática + reglas semánticas), genera automáticamente un compilador completo o partes esenciales de él. Para hacerlo, esos metacompiladores:

- Incorporan internamente las mismas técnicas de Lex/Yacc (por ejemplo, ANTLR tiene su propio algoritmo LL (*) y genera lexers y parsers).
- Pueden usar Lex/Yacc en su implementación (por ejemplo, versiones antiguas de Yacc estaban escritas en C y usaban Lex para su propio análisis).
- Generan código que invoca a esas herramientas o que contiene directamente los analizadores (como hace ANTLR al producir código en Java/Python/C# que incluye la lógica de análisis).

4. Relación de simbiosis

- Un compilador usa Lex/Yacc/ANTLR para no tener que escribir el analizador léxico-sintáctico manualmente.
- Un metacompilador (que es un tipo especial de programa) usa las mismas técnicas y, a menudo, las mismas herramientas para generar compiladores. De hecho, Yacc es un metacompilador que produce un analizador sintáctico.

5. Ejemplo concreto

Imagina que quieres crear un compilador para un lenguaje L.

- Sin metacompilador: escribes manualmente la lógica de análisis semántico y generación de código, pero usas Lex y Yacc para el análisis léxico y sintáctico.
- Con metacompilador (ANTLR): defines la gramática de L, las acciones semánticas y la generación de código en un solo archivo. ANTLR te genera todo el compilador,

incluyendo el analizador léxico y sintáctico automatizados, usando sus propios algoritmos (equivalentes a los de Lex/Yacc pero más modernos).

En ambos casos, el análisis léxico y sintáctico queda automatizado por herramientas basadas en los mismos principios (expresiones regulares + gramáticas). La diferencia es quién invoca esas herramientas: si lo hace el programador del compilador o si lo hace el metacompilador como parte de su proceso de generación. Esto permite cambios rápidos (edita y ejecuta al instante), depuración interactiva (pausa en cualquier punto) y portabilidad (funciona en cualquier máquina con el intérprete instalado), pero es más lento porque reanaliza el código cada vez que se ejecuta una línea, sin optimizaciones globales.

Intérpretes y Arquitecturas Híbridas

Los intérpretes y arquitecturas híbridas priorizan la ejecución dinámica para ofrecer máxima flexibilidad durante el runtime, permitiendo adaptaciones en tiempo real sin necesidad de recompilación completa. Un intérprete es un programa que lee el código fuente línea por línea y lo ejecuta directamente en el momento, sin generar un archivo binario intermedio. En cambio, un compilador analiza todo el código fuente de una vez (fase léxica, sintáctica, semántica y generación de código), lo traduce completo a un binario nativo optimizado (como un .exe en C++) y lo guarda para ejecución posterior.

Las diferencias clave entre un intérprete y un compilador radican en su enfoque de traducción y ejecución del código fuente, impacto en rendimiento, portabilidad y depuración.

- **Producto Generado**

Compilador: Produce un archivo binario portátil (ejecutable standalone) que se puede distribuir y ejecutar sin el código fuente ni el compilador. El código fuente queda "oculto".

Intérprete: No genera archivo ejecutable. Requiere el código fuente y el intérprete en cada ejecución; el código fuente debe estar disponible.

- Velocidad de Ejecución

Compilador: Más rápido (10-20x en promedio), ya que el binario es código máquina optimizado directo para la CPU, sin traducción en runtime.

Intérprete: Más lento, porque traduce cada línea repetidamente en cada ejecución, overhead constante por análisis dinámico.

- Detección de Errores

Compilador: Detecta errores sintácticos y semánticos antes de ejecutar (en fase de compilación), facilitando depuración temprana.

Intérprete: Solo detecta errores durante la ejecución (cuando llega a la línea), lo que permite ejecución parcial, pero complica fixes en runtime.

El compilador prioriza rendimiento y optimización estática para aplicaciones de producción, mientras el intérprete enfatiza interactividad y portabilidad dinámica para desarrollo rápido. Híbridos como JIT fusionan ambos para equilibrar.

La Compilación Just-In-Time (JIT) surgió para combinar lo mejor de ambos mundos: flexibilidad del intérprete + rendimiento del compilador. La compilación JIT representa una arquitectura híbrida donde el código se interpreta inicialmente para un arranque rápido, y luego se compila dinámicamente en código máquina nativo para rutas frecuentemente ejecutadas, optimizando el rendimiento en tiempo real.

Paso a paso cómo funciona JIT:

1. **Fase de interpretación inicial:** Código fuente → Bytecode portable (e.g., .class en Java) → Intérprete lo ejecuta línea por línea para inicio rápido (1-10 ms).
2. **Perfilado dinámico:** El ejecutor cuenta ejecuciones (contadores de bucles, llamadas de funciones) y detecta hotspots (e.g., un bucle que se ejecuta 10,000 veces).
3. **Compilación en caliente:** JIT genera código máquina nativo solo para ese hotspot, aplicando optimizaciones basadas en datos reales (e.g., asume tipos de variables de ejecuciones previas via especulación).
4. **Ejecución híbrida:** Código compilado se cachea; si cambia el contexto, revierte a interpretación.
5. **Beneficios:** +90% rendimiento tras adaptabilidad a patrones runtime.

Ejemplos prácticos:

- **Motor V8 (JavaScript, Chrome/Node.js):** Interpreta JS dinámico inicialmente (Ignition), perfila funciones repetidas (e.g., un render loop en una app web), compila con TurboFan a ARM/x86 optimizado. Soporta WebAssembly vía Liftoff (rápido) + Turbofan (optimizado). Resultado: JS casi tan rápido como C en benchmarks.
- **JVM (Java HotSpot):** Bytecode → intérprete para cold code → C1/C2 JIT para hotspots. Usa "method inlining" y "escape analysis" para eliminar objetos innecesarios, logrando rendimiento nativo tras 1-5 seg de ejecución

WebAssembly (o WASM) es una tecnología que permite ejecutar programas muy rápidos y seguros directamente en navegadores web, servidores o dispositivos pequeños como sensores IoT, sin necesidad de instalar software extra. En el contexto de edge

computing (procesar datos cerca de donde se generan, como en una cámara o un termostato) y IoT (dispositivos conectados del internet de las cosas), WASM actúa como un "ejecutor equilibrado" que combina velocidad con protección, ideal para lugares con poca potencia o conexión lenta.

En Azion Edge, WASM procesa requests HTTP personalizados; EdgeUno usa WASM para IA en dispositivos perimetrales (e.g., detección anomalías en sensores). En IoT, proyectos como Tasmota usan WASM para scripts seguros en ESP32, logrando 10x menos recursos que Lua intérprete. Esto hace WASM el "ejecutor híbrido" perfecto para entornos distribuidos y resource-constrained.

Descompiladores e ingeniería inversa

Ingeniería Inversa

Según Marín (2013), “La ingeniería inversa es el proceso de técnica de descubrir los principios tecnológicos de un producto, herramienta, dispositivo o sistema.” Este descubrimiento se puede realizar mediante el razonamiento abductivo de su estructura, función y operación. Por ejemplo, un niño puede agarrar un dispositivo y desmontarlo, preguntándose así qué elementos hay dentro y cómo su combinación lo hace funcionar.

Del mismo modo un adulto puede realizar este trabajo en un taller con la intención de reparar un sistema electrónico más complejo. Este es un proceso de deconstrucción, de convertir los pasos del desarrollo para analizar y obtener conocimiento de cualquier cosa diseñada y elaborada por un ser vivo. Este tipo de ingeniería busca revelar y determinar los detalles más íntimos, los componentes y sus relaciones o, por lo menos, descubrir cómo se diseñó un objeto en cuestión.

Utilidades de la ingeniería inversa

La principal utilidad que tiene es la de comprender el funcionamiento de los programas; este es un proceso esencial para aprender a programar y desarrollar aplicaciones. Uno de los ejemplos más destacados es el de la investigación de malware, ya que los analistas emplean técnicas de ingeniería inversa para analizar el software malicioso, comprender su funcionamiento y desarrollar las contramedidas eficaces.

Otra de las áreas donde más se ve la ingeniería inversa es en el análisis e investigación de vulnerabilidades, los investigadores de seguridad la utilizan para identificar los puntos débiles del software o sistema para así poder desarrollar parches y actualizaciones abordando estas vulnerabilidades para fortalecer la defensa de la empresa.

No obstante, la ingeniería inversa también tiene algunas aplicaciones que pueden utilizarse con fines maliciosos, como para infringir leyes de propiedad intelectual o derechos de autor, ya que este puede permitir el acceso no autorizado a software propietario o la replicación de tecnología patentada, generando pérdidas a los creados y titulares de los derechos de autor.

También puede ser utilizada por delincuentes de ciberseguridad para comprender como es el funcionamiento interno de los sistemas de software para acceder a los sistemas de servidor sin una debida autorización. Esto pone a la ingeniería inversa en una gama gris de utilidades; pero, como funciona en la mayoría de los casos: Una herramienta no está creada para el mal o para el bien, es quién la utiliza el que decide.

Ingeniería inversa en el área del software

Se puede utilizar en diferentes áreas para cubrir todo tipo de usos variados, como puede serlo el control de calidad de las funciones del software, el análisis de los protocolos de

comunicación de red, para la detección de virus y programas maliciosos, para la portabilidad y mantenimiento de abandonware y la mejorabilidad de las compatibilidades de software con las plataformas y software de terceros.

También puede ser utilizada para analizar los productos de la competencia, aunque muchas empresas prohíben la ingeniería inversa en sus productos y lo incluyen en las condiciones de licencia, no obstante, la vía legal no aplica al análisis de protocolos; del mismo modo que muchas de esas cláusulas no tienen validez en otros países. Una vez que se adquiere un software, se tiene derecho a someterlo a una ingeniería inversa para comprobar la seguridad de la aplicación y solucionar fallos.

Fases de la ingeniería inversa de software

Se puede dividir en dos etapas principales, la primera de ellas se considera una observación a gran escala utilizando herramientas y varios servicios del sistema operativo para determinar la estructura general. La segunda etapa es más profunda, pues se orienta a fragmentos de código para comprenderlos en su estructura y funcionalidad.

De una manera más detallada, se puede resumir en 3 pasos:

1. Extracción de información: En esta fase inicial se recopilan los datos posibles sobre el software sin ejecutarlo, es decir, es un análisis estático. Se identifican los tipos de archivos, las herramientas de compilación, los recursos incrustados, posteriormente el ejecutable se introduce en un desamblador para convertir el código máquina en lenguaje ensamblador y se utiliza un descompilador para transformar el código compilado en un lenguaje de alto nivel más comprensible.
2. Análisis dinámico y observación: En esta fase se ejecuta el programa en un entorno controlado para observar su comportamiento. Se usa un depurador para ejecutar el código línea por línea, establecer puntos de interrupción en puntos

críticos e inspeccionar el estado del programa. Se usan herramientas para capturar y analizar el tráfico de red generado por la aplicación, siendo esto esencial para comprender los protocolos de comunicación.

3. **Reconstrucción:** Con todos los datos obtenidos se reconstruye completamente la lógica y diseño del software, se documentan minuciosamente y este conocimiento se traduce en medidas de seguridad viables, como el desarrollo de parches, la creación de firmas de detección o la implementación de las nuevas prácticas de codificación defensiva.

Herramientas esenciales para la ingeniería inversa

Esta práctica utiliza muchas herramientas específicas para su funcionalidad para poder traducir las instrucciones de la máquina en lógica comprensible para los humanos y poder observar el comportamiento del programa en un entorno controlado. Una de estas herramientas son los desensambladores, que desarrolla un proceso contrario al ensamblador y que será diferente dependiendo de la plataforma en la cual se utilice.

Descompiladores

Un descompilador es una herramienta de software diseñada para realizar ingeniería inversa en programas informáticos compilados o archivos binarios en un lenguaje de programación legible por los humanos, como puede serlo C o Java. Estos se usan para analizar y comprender la funcionalidad de una aplicación y, a menudo, para identificar y corregir errores o recuperar el código fuente perdido.

Un proceso de descompilación implica revertir el proceso de compilación, analizando el código compilado y reconstruyendo el código fuente original. Esto se realiza a través de

interpretar el código binario e identificar los patrones y estructuras característicos del código original. El descompilador depende del lenguaje y la plataforma del código ejecutable.

Los pasos suelen ser siempre iguales, el desmontaje donde el descompilador lee el código ejecutable y crea una representación de bajo nivel de las instrucciones del código, analiza el flujo de datos y reconstruye el código fuente original. Suele complicarse por factores como las optimizaciones del compilador y las técnicas de ofuscación diseñadas para que el código sea más difícil de descompilar.

AST como Lenguaje Universal

A pesar de que sus objetivos en el ciclo de vida del software son distintos (generar, ejecutar o revertir código), los descompiladores, metacompiladores e intérpretes se unen en una pieza central y universal: el Árbol de Sintaxis Abstracta (AST). El AST es una "simplificación de árbol sintáctico que represente toda la información necesaria para el resto del procesamiento del programa de un modo más eficiente" (Ortín et al., 2004), su gran valor radica en que representa la esencia de las operaciones lógicas, eliminando detalles léxicos innecesarios que son propios de la sintaxis concreta, tales como espacios, comas, palabras reservadas o paréntesis (SBScript, s.f.). Al hacer esto, el AST preserva la intención jerárquica y lógica pura del programador en un formato de datos altamente procesable.

Esta estructura tiene usos específicos y adaptados según el flujo de la herramienta que lo utilice:

- Metacompilación y compilación (flujo hacia adelante): Durante el proceso de generación a partir de descripciones de lenguajes, el AST se utiliza como la base matemática estructural. En lugar de procesar texto crudo, el compilador anota los nodos de este árbol para validar las reglas gramaticales y semánticas complejas, como la validación de tipos o el ámbito de las variables (Ortín et al., 2004). Una vez que el AST está completamente validado y decorado, sus reglas se transforman paso a paso en representaciones intermedias o código ejecutable.
- Interacción técnica en intérpretes (flujo en tiempo real): Por su parte, los intérpretes dependen del AST para lograr una ejecución dinámica. En lugar de traducir el programa entero por adelantado, el intérprete parte del AST y lo recorre instrucción por instrucción "junto con los datos de entrada, para obtener los resultados" en tiempo real (Universitat Jaume I, s.f.). Para lograr esta ejecución directa y añadir anotaciones semánticas eficientemente, la arquitectura moderna implementa recorridos estructurados a través de patrones de diseño orientados a objetos, destacando fuertemente el uso del patrón visitante (*Visitor pattern*) para separar las operaciones de la estructura misma del árbol (Ortín et al., 2004).
- Descompilación (flujo hacia atrás): En el contexto de la ingeniería inversa, el proceso de traducción es estrictamente opuesto. Un descompilador se enfrenta al código máquina incomprensible e intenta deducir representaciones intermedias (IR) y árboles abstractos desde abajo hacia arriba (Universitat Jaume I, s.f.). Al reconstruir un AST a partir de flujos de código binario, la herramienta es capaz de inferir la estructura lógica de alto nivel original, lo que permite al analista humano entender la intención inicial del programa.

La existencia de esta estructura común y estandarizada hace que la manipulación matemática del programa sea viable independientemente de la dirección del flujo de

información. Sin el Árbol de Sintaxis Abstracta operando como puente de lenguaje universal, la interconexión teórica de estas herramientas modernas sería imposible.

Hibridación con Llms y Modelos Fundacionales

En la actualidad, la integración de grandes modelos de lenguaje (*Large Language Models* o LLMs) ha transformado radicalmente las herramientas clásicas de traducción de lenguajes, convirtiéndolas en ecosistemas híbridos. En este nuevo paradigma, el software tradicional sigue encargándose del formalismo matemático y la corrección sintáctica estricta, mientras que la inteligencia artificial (IA) aporta una potente capa de probabilidad semántica capaz de interpretar y reconstruir la intención original detrás del código (Jelodar et al., 2025).

Ingeniería Inversa Asistida

Históricamente, la descompilación ha sufrido un severo problema de "entropía informativa". Al transformar un lenguaje de alto nivel a código binario o ensamblador, la información semántica vital (como los nombres de las variables, las funciones y los comentarios) se pierde de forma irreversible (Tan et al., 2024). Esto hace que la ingeniería inversa tradicional devuelva un código funcional, pero extremadamente difícil de interpretar (García, 2024).

Hoy en día, los LLMs actúan como motores avanzados de inferencia contextual. Modelos especializados como *LLM4Decompile*, entrenados con miles de millones de tokens, analizan el flujo de datos dentro del código desensamblado para "adivinar" o inferir con alta precisión los nombres originales y las estructuras lógicas perdidas (Tan et al., 2024). Esta capacidad de recuperación mejora drásticamente la legibilidad humana, reduciendo el tiempo necesario para la auditoría de *malware*, el descubrimiento de vulnerabilidades y el análisis de software legado (García, 2024; Jelodar et al., 2025).

Optimización y Aplicación de Autotuning

En el ámbito de la generación y optimización, el avance más significativo es la aparición de modelos fundacionales dedicados a emular el comportamiento de un compilador. Un ejemplo contundente es el *Meta LLM Compiler*, entrenado con un corpus masivo de 546 billones de tokens de representaciones intermedias (LLVM-IR) y código ensamblador (Cummins et al., 2024).

- *Eficiencia en autotuning:* El modelo ha demostrado ser capaz de alcanzar un 77 % del potencial óptimo de *autotuning*, optimizando automáticamente el tamaño del código y la velocidad de ejecución (Cummins et al., 2024).
- *Precisión round-trip:* En tareas de ingeniería inversa, estos modelos logran un éxito del 45 % en el proceso de ida y vuelta (*round-trip*), una cifra récord en la capacidad de ir desde un ensamblador de bajo nivel hacia representaciones intermedias y volver al inicio sin romper la funcionalidad (Cummins et al., 2024).

Ecosistemas Híbridos

Herramientas actuales de la industria representan el estándar de desarrollo híbrido moderno. En estos ecosistemas, la IA asume el rol de generador creativo. Sin embargo, estudios recientes revelan que los modelos de lenguaje pueden sufrir de "alucinaciones" y sugerir código que introduce vulnerabilidades de seguridad (García, 2024; Jelodar et al., 2025). Por ello, la delegación es crítica. Mientras que la IA sugiere optimizaciones audaces, la validación final se delega siempre a los compiladores tradicionales. Estos garantizan la corrección estructural y validan de manera formal que el software final sea completamente funcional y seguro (Jelodar et al., 2025).

Soberanía Tecnológica y Supply-chain Attacks

La unificación de las herramientas de traducción de lenguajes es vital para enfrentar los complejos desafíos geopolíticos y de seguridad actuales. En un escenario global donde las dependencias de software pueden ser explotadas, la protección de las infraestructuras críticas se ha convertido en una prioridad estratégica (Llorente, 2025).

El Hito del Bootstrapping

El *bootstrapping* (el proceso histórico mediante el cual un compilador es capaz de compilar su propio código fuente) es un pilar fundamental para garantizar la trazabilidad exhaustiva y la soberanía tecnológica. Lograr este hito permite a las organizaciones independizarse de cadenas de suministro lideradas por terceros. Al utilizar lenguajes modernos que cuentan con compiladores autocontenidos y rigurosamente auditables, las organizaciones pueden asegurar que no existan manipulaciones ocultas. Este rigor conceptual es heredero directo de la histórica advertencia realizada por Ken Thompson en su discurso del Premio Turing, "*Reflections on Trusting Trust*", donde demostró que un compilador puede ser subvertido para insertar caballos de Troya encubiertos, confirmando que, sin herramientas auditables, ninguna revisión visual del código fuente es suficiente.

Defensa contra Ataques a la Cadena de Suministro

Debido a la interconexión de la industria actual, los ataques a la cadena de suministro (*supply-chain attacks*) se han convertido indiscutiblemente en la "nueva puerta de entrada a las organizaciones" (INSSIDE, 2025). Los cibercriminales inyectan código malicioso en binarios o librerías durante su desarrollo, comprometiendo al eslabón más débil de la cadena de confianza. Casos de extrema gravedad mundial, como el de SolarWinds, han evidenciado que vulnerar a un solo proveedor de software legítimo basta para infectar a miles de infraestructuras en simultáneo (Clay, 2026; INSSIDE, 2025).

La solución moderna para neutralizar este vector de ataque radica en un enfoque combinado:

- Compiladores auditables: Son indispensables para garantizar que el proceso de creación sea completamente transparente, utilizando técnicas como la "doble compilación diversa" para certificar que el ejecutable no ha sido víctima de una infección de compilador (Wheeler, 2009).
- Descompiladores modernos: Son la principal línea de defensa para realizar auditorías de caja negra sobre *firmwares* de terceros cuando no se dispone del código fuente. Dado que los atacantes han logrado modificar *firmwares* y firmarlos con certificados fraudulentos (Becolve Digital, 2019), la ingeniería inversa permite verificar la autenticidad estructural del código, previniendo la manipulación de sistemas críticos e IoT.

Referencias

- Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2006). *Compilers: Principles, techniques, and tools* (2nd ed.). Addison-Wesley. (Texto fundamental sobre el diseño de compiladores, que sienta las bases teóricas para entender los metacompiladores).
- Johnson, S. C. (1979). YACC: Yet another compiler-compiler. En *UNIX programmer's manual* (Vol. 2b). Holt, Rinehart and Winston. (Publicación original del generador de analizadores sintácticos Yacc, un ejemplo clásico de metacompilador basado en gramáticas).
- Parr, T. (2014). *The definitive ANTLR 4 reference* (2nd ed.). The Pragmatic Programmers. (Guía de referencia sobre ANTLR, un metacompilador moderno de propósito general que genera analizadores léxicos y sintácticos).
- Thurston, A. (2006). *Ragel state machine compiler*. Recuperado de <http://www.colm.net/open-source/ragel/> (Herramienta que compila máquinas de estados finitos a partir de expresiones regulares, usada en el análisis léxico).
- Levine, J. R., Mason, T., & Brown, D. (1992). *Lex & yacc* (2nd ed.). O'Reilly & Associates. (Guía práctica sobre el uso de Lex y Yacc para construir analizadores).
- Becolve Digital. (12 de febrero de 2019). *Una nueva solución para evitar “Supply Chain Attacks” y asegurar la integridad del firmware/software que se instala en entonos OT / Críticos*. Becolve Digital. <https://becolve.com/blog/una-nueva-solucion-para-evitar-supply-chain-attacks-y-asegurar-la-integridad-del-firmware-software-que-se-instala-en-entonos-ot-criticos/>

Clay, J. (22 de abril de 2026). *¿Qué es un ataque a la cadena de suministro?* Trend Micro.

https://www.trendmicro.com/es_es/what-is/cyber-attack/supply-chain-attack.html

Concrete and Abstract Syntax. (s.f.). *Principles of Programming Languages*.

https://bguppl.github.io/interpreters/practice_sessions/ps4.html

Cummins, C., Seeker, V., Grubisic, D., Rozière, B., Gehring, J., Synnaeve, G. y Leather, H.

(2024). *Meta Large Language Model Compiler: Foundation Models of Compiler*

Optimization. Meta AI / arXiv. <https://arxiv.org/html/2407.02524v1>

García, D. (2024). *Ingeniería inversa asistida por inteligencia artificial* [Memoria del trabajo

de fin de grado, Universitat Politècnica de Catalunya]. UPCommons.

<https://upcommons.upc.edu/server/api/core/bitstreams/32d214bd-5d17-401c-b5de-387aa2f306fd/content>

INSSIDE. (2025). *Supply Chain Attacks: la nueva puerta de entrada a las organizaciones*.

<https://www.insside.net/cl/supply-chain-attacks-la-nueva-puerta-de-entrada-a-las-organizaciones/>

Jelodar, H., Meymani, M. y Razavi-Far, R. (2025). *Large Language Models (LLMs) for*

Source Code Analysis: applications, models and datasets. arXiv.

<https://arxiv.org/html/2503.17502v1>

Llorente, P. (2025). *El reto de la soberanía tecnológica: hacia un ecosistema digital europeo*

propio. Real Instituto Elcano. [https://www.realinstitutoelcano.org/analisis/el-reto-de-](https://www.realinstitutoelcano.org/analisis/el-reto-de-la-soberania-tecnologica-hacia-un-ecosistema-digital-europeo-propio/)

[la-soberania-tecnologica-hacia-un-ecosistema-digital-europeo-propio/](https://www.realinstitutoelcano.org/analisis/el-reto-de-la-soberania-tecnologica-hacia-un-ecosistema-digital-europeo-propio/)

Ortín, F., Cueva, J., Luengo, M., Juan, A., Labra, J. e Izquierdo, R. (2004). *Análisis*

semántico en procesadores de lenguaje. Universidad de Oviedo [Archivo PDF].

<https://www.reflection.uniovi.es/ortin/publications/semantico.pdf>

SBScript. (s.f.). *Compiladores 2*. <https://esvux.github.io/SBScript/ast.html>

Tan, H., Luo, Q., Li, J. y Zhang, Y. (2024). *LLM4Decompile: Decompile Binary Code with Large Language Models*. arXiv. <https://arxiv.org/html/2403.05286v3>

Universitat Jaume I. (s.f.). *II26 Procesadores de lenguaje. Estructura de los compiladores e intérpretes* [Archivo PDF].

<https://hopelchen.tecnm.mx/principal/sylabus/fpdb/recursos/r91607.PDF>

Wheeler, D. A. (2009). *Fully Countering Trusting Trust through Diverse Double-Compiling* [Archivo PDF]. <https://dwheeler.com/trusting-trust/dissertation/wheeler-trusting-trust-ddc.pdf>

Intérpretes vs. Compiladores (A3C34A2D05):

<https://opencontent.upct.es/f0fc64e3b2d34f3a872a841a9c37b249/6aedb53b964646858a769a9ce93da286/>

Glosario de tecnología: qué significa Diferencia entre compilador e intérprete:

<https://www.infobae.com/tecnologia/2025/04/25/glosario-de-tecnologia-que-significa-diferencia-entre-compilador-e-interprete/> (25 abr 2025)

Qué es un Compilador: Principales Funciones: <https://immune.institute/blog/que-es-un-compilador/> (19 abr 2023)

Interprete vs compilador (Slideshare): <https://es.slideshare.net/slideshow/interprete-vs-compilador/71744279> (3 feb 2017)

El compilador JIT (IBM Docs): <https://www.ibm.com/docs/es/sdk-java-technology/8?topic=reference-jit-compiler>

Motor V8: el motor de código abierto para JavaScript y WebAssembly:

<https://pablogarciajc.com/blog/frontend/motor-v8-el-motor-de-codigo-abierto-para-javascript-y-webassembly/> (20 dic 2025)

Cómo el compilador JIT de Java mejora la velocidad de ejecución:

<https://www.victoriglesias.net/como-el-compilador-jit-de-java-mejora-la-velocidad-de-ejecucion/> (3 feb 2022)

La solución multiplataforma para Edge Computing e IA (EdgeUno):

<https://edgeuno.com/es/webassembly-the-multi-platform-solution-for-edge-computing-and-ai/> (29 may 2024)

WebAssembly en la Azion Web Platform: hazlo todo en el edge:

<https://www.azion.com/es/blog/webassembly-en-la-plataforma-de-azion-hazlo-todo-en-el-edge/> (2 nov 2022)

Considering WebAssembly Containers for Edge Computing on IoT: [https://www.diva-](https://www.diva-portal.org/smash/get/diva2:1451494/FULLTEXT02.pdf)

[portal.org/smash/get/diva2:1451494/FULLTEXT02.pdf](https://www.diva-portal.org/smash/get/diva2:1451494/FULLTEXT02.pdf)

WasmEdge: WebAssembly runtime for edge: <https://wasmedge.org>